

1. INTRODUCTION TO ARTIFICIAL NEURAL NETWORKS

Artificial Neural Networks (ANNs) are computing systems inspired by biological neural networks. The core building block of an ANN is the artificial neuron, also known as a Perceptron.

A single perceptron takes multiple input signals ($x_1, x_2, \dots, x_n$), multiplies each by an associated scalar weight ($w_1, w_2, \dots, w_n$), adds a real-valued bias term b , and passes the weighted sum through a non-linear activation function ϕ .

Mathematical representation of the raw weighted sum (z):

$$z = (w_1 * x_1) + (w_2 * x_2) + \dots + (w_n * x_n) + b$$

Or in vector notation: $z = \text{transpose}(W) * X + b$

The final scalar output (a) is computed as:

$$a = \phi(z)$$

2. ACTIVATION FUNCTIONS AND NON-LINEARITY

Without non-linear activation functions, a neural network with arbitrary depth (L layers) collapses mathematically into a single linear regression model. Activation functions allow neural networks to approximate non-linear decision boundaries.

A. SIGMOID ACTIVATION

Formula: $\phi(z) = 1 / (1 + \exp(-z))$

- Range: (0, 1)
- Primary Use: Binary classification output layer.
- Limitations: Saturated neuron outputs cause vanishing gradients during backpropagation because the maximum derivative value is 0.25 at $z = 0$.

B. RECTIFIED LINEAR UNIT (ReLU)

Formula: $\phi(z) = \max(0, z)$

- Range: [0, infinity)
- Primary Use: Default activation choice for hidden layers in modern deep architectures.
- Advantages: Extremely fast computationally; does not saturate in the positive domain.
- Limitations: "Dying ReLU" problem occurs when large negative gradients push neuron weights into a state where they never activate on any future dataset example.

C. LEAKY ReLU

Formula: $\phi(z) = \max(\alpha * z, z)$ where α is a small constant such as 0.01.

- Prevents dead neurons by ensuring a small non-zero gradient for negative z values.
-

3. FORWARD PROPAGATION IN MULTI-LAYER PERCEPTRONS (MLPs)

Consider a 3-layer neural network with input vector X , hidden layer 1 activations A_1 , hidden layer 2 activations A_2 , and final output $\hat{Y}$.

Layer 1 Computation:

$$Z_1 = W_1 * X + B_1$$

$$A_1 = \text{ReLU}(Z_1)$$

Layer 2 Computation:

$$Z_2 = W_2 * A_1 + B_2$$

$$A_2 = \text{ReLU}(Z_2)$$

Output Layer Computation:

$$Z_3 = W_3 * A_2 + B_3$$

$$\hat{Y} = \text{Softmax}(Z_3)$$

LABORATORY EXERCISE FOR CLASS:

Implement forward propagation for a 2-hidden layer network from scratch in Python using only NumPy functions. Verify matrix dimension consistency for a batch size of 32 examples.